

# Memory Is the Price: Evidence from the LLM Serving Market on the Economics of Mixture-of-Experts Inference

Michael J. Yuan, PhD\*

Ju Long, PhD†

Working paper · Draft of July 2026

Data collected 2026-07-06

## Abstract

Sparse Mixture-of-Experts (MoE) models are widely claimed to cost what their *active* compute costs. We test this claim against the LLM serving market itself. Because providers disclose almost nothing about their production systems, we treat posted prices as an observable proxy for the latent serving configuration. From OpenRouter’s public endpoints API we assemble a snapshot (2026-07-06) of every open-weight, general-purpose model served by at least three providers: 46 models, of which 38 form the primary panel of 289 provider–model observations from 42 providers, with architecture verified against each model’s HuggingFace repository. The DeepSeek panels are excluded from the primary specification because the vendor anchors its own serving prices, and are analyzed separately. The prices contradict the compute-pricing claim. A hedonic log-log regression of median output price on active parameters and the sparsity ratio yields a sparsity elasticity of 0.481 ( $t = 4.9$ ,  $p = 1.9 \times 10^{-5}$ ), 84% of the active-parameter elasticity; the regression is jointly significant with  $F(2, 35) = 16.8$  ( $p = 7.7 \times 10^{-6}$ ) and  $R^2 = 0.49$ . MoE models are therefore billed most of the way toward their total memory footprint (the size of the whole model in memory, including every expert), and customers keep little of the advertised compute discount. Market structure supports the scale interpretation. The price floor of every panel of 200B+ total parameters belongs to a vertically integrated provider, a neocloud in eleven of seventeen panels and the model’s own vendor in six, and hyperscalers price 1.2–8.8× above the floor and never set it. MoE price dispersion remains compressed relative to dense models (median coefficient of variation 0.23 versus 0.35), consistent with selective listing. DeepSeek is the revealing exception. Its sparse-attention generation is served at roughly one fifth of the price its size would otherwise command, and the vendor itself sets the floor beneath its flagship. This is the vertical-integration mechanism in its purest form.

## 1 Introduction

Between 2024 and 2026, the architecture of frontier open-weight language models changed decisively. Mixtral 8x7B (December 2023) reintroduced the sparse Mixture-of-Experts (MoE) design to open models with 8 experts per layer, of which 2 are activated per token. DeepSeek-V3 and R1 (December 2024 / January 2025) scaled the recipe to 671 billion total parameters with only 37 billion active per token (256 routed experts, top-8 routing) [1]. Within eighteen months, essentially every major open-weight release followed: Llama 4 Scout and Maverick (16 and 128 experts, top-1) [2, 3], Qwen3-235B (128

---

\*ByteFuture Inc.

†Texas State University

experts, top-8) [4], Kimi K2 (one trillion parameters, 384 experts) [5], GPT-OSS-120B (128 experts, top-4) [6], GLM-4.5 (160 experts) [7], and MiniMax M2 (256 experts) [8]. The sparsity ratio (total to active parameters) has grown from  $\sim 4\times$  in Mixtral to  $18\text{--}31\times$  in the current frontier. Meanwhile the largest open *dense* models have retreated: Llama-3.1-405B, the largest dense open release, is effectively delisted from the commercial serving market as of mid-2026.

Sparsity is marketed as an efficiency technology. The paper that popularized the design promised “outrageous numbers of parameters” at “a constant computational cost” [9], and on compute-based reasoning a 671B-parameter model that computes with only 37B parameters per token should cost about as much to serve as a 37B dense model [10, 11]. Market prices do not bear this out. At the time of our data collection, Kimi K2.6 (one trillion total parameters, 32B active) commanded a median output price of \$3.55 per million tokens across twenty providers, while Llama-3.3-70B, a dense model with more than twice K2.6’s active parameters, cost \$0.72. Per unit of active compute, the MoE model is roughly eleven times more expensive. A second irregularity compounds the first: prices for the *same* model differ substantially across providers. Google lists Qwen3-235B-A22B at \$0.88 per million output tokens while DeepInfra lists it at \$0.10, a spread of nearly nine times on identical weights.

These patterns cannot be explained from provider disclosures, because there are almost none. The production systems behind commercial inference endpoints are, with rare exceptions, undisclosed. Hyperscaler model catalogs publish prices, context windows, regions, and quotas, but not hardware, served precision, parallelism, or cluster scale; Google’s managed-Llama announcement states simply that “the underlying infrastructure, scaling, and management costs are incorporated into the API usage price” [12]. On OpenRouter, the marketplace whose data we use, the per-endpoint schema contains no field for hardware or deployment topology, and even the self-reported quantization tag is frequently “unknown” [13]. The opacity extends to the served model itself: a black-box audit found that 11 of 31 commercial Llama endpoints produced output distributions deviating from Meta’s reference weights, because providers “may quantize, watermark, or finetune the underlying model... possibly without notifying users” [14].

The secrecy is not uniform, and the exceptions delimit what is actually hidden. The serving software stack is open source (vLLM, SGLang), and the techniques for efficient MoE serving are published. DeepSeek, a model vendor with an evident strategic interest in commoditizing the serving of its own models, has disclosed its production system in detail, down to its EP144 decode topology and a theoretical 545% margin [15]. Third parties have publicly reproduced that recipe [16]. What remains hidden is the mapping from any *particular* commercial endpoint to its deployment: which provider runs expert parallelism at what scale, at what batch size and utilization, at what unit cost. Engineering blogs occasionally reveal fragments (Cloudflare, for instance, has described the multi-node system behind its largest models [17]), but for the 42 providers in our panel, the economically decisive parameters are unobservable.

The one thing every provider must publish is a price. This turns an information-asymmetry problem into an inference problem with a classical structure. The serving configuration behind an endpoint (cluster scale, parallelism degree, batch size, served precision, utilization) is the provider’s private information; the observer sees only the posted price. But prices are the market’s mechanism for aggregating dispersed private information [18]. Open-weight serving, moreover, approximates the textbook competitive case: the weights are free, the software is open source, entry is unrestricted, and dozens of providers quote side by side on a public marketplace. Competition disciplines posted prices toward marginal cost, and marginal cost is a function of the serving technology chosen. Sustained price structure therefore serves as an observable proxy for the latent cost structure and, through it, for the latent hardware configuration that generates that cost structure. Our empirical strategy follows

the hedonic-pricing tradition [19]: we regress prices on the product characteristics that drive resource consumption (active parameters; total memory footprint) and read the estimated coefficients as implicit prices, the market’s marginal valuation of each underlying resource. The industry’s claim that sparsity’s compute savings pass through into proportionally cheaper serving then becomes a testable restriction on hedonic coefficients (H1). The question of what serving scale is required to capture those savings becomes a prediction about which provider class posts each panel’s price floor, who participates in each market, and how endogenous participation truncates the observed price distribution (H2).

We are explicit about the limits of this inversion. Price is a low-dimensional proxy: it cannot reconstruct any single endpoint’s configuration, and an individual quote confounds cost with strategy (loss leaders, price discrimination, enterprise positioning). Our inferences accordingly rest on moments of the price distribution that are difficult to generate strategically: elasticities estimated across 38 models, price floors replicated across seventeen independent panels, and dispersion patterns across 42 providers.

Three strands of prior work frame our contribution. Bottom-up cost models compute what MoE serving *should* cost under optimal deployment [20, 21] but do not test market prices. Black-box measurement studies characterize hosted endpoints behaviorally. Gao et al. detect undisclosed weight modifications from outputs [14]. Li et al. document that hosted open-weight APIs are heterogeneous services whose listed prices are their most anchored attribute, observing descriptively, without a regression, that neither total nor active parameter count suffices to explain price [22]. Closest to our first hypothesis, Scher compared five MoE models against a dense price trendline in early 2025 and observed informally that MoE prices sit “somewhere between their active and total parameter count, perhaps closer to the total parameter count” [23]. We formalize that observation into an identified quantity. To our knowledge, this paper makes three contributions. It provides the first hedonic estimate of the sparsity elasticity of price (§4.1). It presents the first evidence that the price floors for large MoE models are set exclusively by massively parallel providers (§4.2). And it provides the first documentation that provider participation, and through selection the measured price dispersion, is gated by model memory footprint (§4.2–4.3). The concluding section draws out the hardware implication.

## 2 Background and hypotheses

### 2.1 Background: the cost structure of decoding

Transformer inference has two phases. *Prefill* processes the prompt in parallel and is compute-bound. *Decode* generates output tokens one at a time; each step must read the model’s weights from memory while performing only  $\sim 2$  floating-point operations per parameter read, making it memory-bound on all practical hardware. Output-token prices therefore reflect decode economics, and we study output prices throughout.

Two facts about the input markets frame our hypotheses. First, high-bandwidth memory (HBM) is the expensive and supply-constrained component of AI accelerators. HBM3E commanded a price premium more than four times that of DDR5 in 2025 [24] and represents roughly 40–50% of an accelerator’s manufacturing cost [25]. SK hynix, the leading supplier, sold out its capacity into 2026 [26]. Second, the number of GPUs a provider must deploy for a model is determined by the model’s *total* memory footprint, not by its compute. DeepSeek R1 at FP8 requires  $\sim 700$  GB for weights, making an  $8 \times H200$  node (1,128 GB) the standard minimal deployment [27]; Kimi K2 at FP8 fills the same node almost completely. Sparsity does not reduce this footprint: although only  $k$  of  $E$  experts fire per token, all  $E$  must reside in memory, because different tokens in a batch activate different experts [21].

A third fact concerns serving efficiency at scale. On a single 8-GPU node, a 256-expert model

cannot be batched to compute efficiency. With top-8 routing, each expert receives only  $K/32$  tokens per step from a batch of  $K$ , far below the  $\sim 128$ -token tile that saturates the grouped-GEMM kernels. The node therefore remains memory-bound at well under 1% model-FLOPS utilization at any feasible batch [28, 29]. Recovering efficiency requires *expert parallelism* (EP): spreading experts across many GPUs and aggregating tokens from thousands of concurrent users so that every expert sees full tiles. Published EP deployments include SGLang’s  $96 \times \text{H100}$  system [16] and DeepSeek’s own production units of EP144 across 18 nodes [15].

## 2.2 Hypothesis 1: sparsity makes inference cheap (the compute-pricing view)

The proposition that MoE reduces serving cost by reducing compute has accompanied the architecture since its introduction, stated by its strongest proponents. Switch Transformers promised models “with outrageous numbers of parameters” at “a constant computational cost” [9]. Mistral’s Mixtral 8x7B announcement drew the pricing implication explicitly: the model “only uses 12.9B parameters per token” and “therefore, processes input and generates output at the same speed and for the same cost as a 12.9B model” [10]. Databricks made the same argument for DBRX, crediting “about half as many active parameters” for inference “up to 2x faster than LLaMA2-70B” [11]. Taken at face value, the claim yields a sharp prediction about market prices: a model’s output price is determined by its active parameters, and the sparsity ratio is free to the customer. We adopt this prediction as our first hypothesis. Formally, in the hedonic price regression

$$\log(\text{price}) = c_0 + c_1 \log(\text{active\_params}) + c_2 \log(\text{total}/\text{active}) \quad (1)$$

the coefficients are implicit prices in Rosen’s sense [19]. The coefficient  $c_1$  is the market’s marginal valuation of active compute, and  $c_2$  is its valuation of resident-but-inactive parameters, the capacity that must be held in memory although it does not fire. *H1 (compute pricing)* predicts  $c_2 = 0$ . The alternative follows from §2.1. If memory rather than compute is the first-order cost of serving, because capital tied up in HBM scales with bytes held and decode throughput is limited by bytes streamed, then prices should track the *total* footprint. This implies  $c_2 \approx c_1$ : a 671B/37B MoE is billed like a 671B dense model. The ratio  $c_2/c_1$  therefore measures the share of the advertised sparsity discount that the market withholds from customers. Under H1 the ratio is zero, because the full compute saving passes through to prices; under the memory-pricing alternative it approaches one, and customers keep none of the discount.

## 2.3 Hypothesis 2: capturing the compute savings requires scale

While H1 concerns the composition of serving cost (which resource the market prices), our second hypothesis concerns scale economies: the minimum efficient scale at which a provider can realize the compute savings of sparsity. The mechanism of §2.1 implies that these savings, whatever share of them reaches customers, are only captured when enough concurrent traffic is aggregated that every expert sees full tiles, and that requires dedicated, disaggregated decoding infrastructure far larger than a single node. The serving-systems literature has converged on exactly this design. Splitwise showed that splitting the prefill and decode phases onto separate machine pools yields  $1.4\times$  throughput at 20% lower cost, observing that the decode phase “does not require the compute capability of the latest GPUs” [30]. DistServe disaggregates the phases onto different GPUs for up to  $7.4\times$  goodput [31]. Moonshot’s production platform for Kimi, Mooncake, “separates the prefill and decoding clusters” outright [32]. For large MoE, the decode pool must additionally be expert-parallel, spread across tens

to hundreds of GPUs [16, 15, 33]. The economic consequence is H2: the ability to serve large MoE at competitive prices is confined to providers who operate such pools and can amortize them across massive request volume. This predicts two observable market patterns. (a) The *price floor* for each large-MoE model should be set by a massively parallel, vertically integrated provider, a specialized high-volume GPU cloud (“neocloud”) or the model’s own vendor, rather than by a general-purpose cloud. (b) General-purpose clouds, whose business does not justify dedicated EP pools per open model, should price visibly above the floor. A corollary concerns price *dispersion*: providers whose latent cost exceeds the competitive price tend not to list, so the measured spread of prices for large MoE should be *compressed* relative to dense models that any provider can serve. We deliberately do not use raw participation counts as a test: how many providers list a model is confounded by demand, because newer and more capable models attract providers regardless of their size.

### 3 Method and data

#### 3.1 Price panel

We collected the complete endpoint listings for open-weight models from OpenRouter’s public endpoints API on 2026-07-06, the data source underlying its public model pages [13, 34]. The candidate universe is mechanical rather than curated, and the construction is a five-step funnel that the collection script (`collect_snapshot.py`) and analysis script both reproduce. First, every listed model with a HuggingFace weights repository qualifies as a candidate, excluding free-tier variants and code-specialized models; 129 candidates were fetched, yielding 532 raw endpoint rows. Second, zero-price and deranked or offline endpoints are dropped (478 rows remain). Third, the unit of observation is one quote per provider per model, keeping the cheapest, which is the price a customer comparing providers would face (466 rows). Fourth, models with fewer than three qualifying providers are dropped, leaving 50 panels, and four of those are removed by category rules: two vision-language variants, one community merge, and one alias listing that points to the same weights as a versioned listing. The result is 46 qualifying panels comprising 347 provider–model observations, spot-verified against provider price pages (§3.3). The qualifying universe itself documents the architectural shift described in §1: 36 of the 46 models are Mixture-of-Experts and only 10 are dense.

The primary specification excludes the eight DeepSeek panels. The exclusion is on identifying-assumption grounds stated in §3.4, not on fit. DeepSeek is the one vendor that both anchors the market price of its own models and publishes its serving stack to third parties. Its panels therefore do not measure the competitive third-party serving process that identifies cost structure. Section 5 compares the DeepSeek panels against the prices predicted by the regression fitted to the other models. The primary panel is **38 models (28 MoE spanning expert-to-activation ratios  $E/k$  from 4 to 128, and 10 dense) comprising 289 observations from 42 distinct providers.**

#### 3.2 Architecture data

For each model we recorded total parameters, active parameters per token, expert count  $E$ , and routing top- $k$ , verified against the model repository’s HuggingFace `config.json` and model card (fetched at collection time) [2, 3, 4, 5, 6, 7, 8, 35]. The archived architecture file records the repository for every panel model. Two verification notes apply. First, safetensors parameter counts are unreliable for quantization-packed checkpoints, so official model-card figures take precedence; DeepSeek V4 Flash, for example, is 284B total parameters by its card although its packed checkpoint stores fewer tensor

elements. Second, MiniMax M3 does not publish its expert count, so it is excluded from the analyses that require  $E/k$ . Dense models take active = total and  $E/k = 1$ .

### 3.3 Verification pass

Two mechanical rules produce the final panel from the raw rows, and both are checkable against the archived data: the one-quote-per-provider rule and the three-provider threshold stated above. In addition, the panel’s single most extreme value, DeepInfra’s \$0.10/M output price on Qwen3-235B-A22B, roughly eight times below its panel median, was confirmed on DeepInfra’s own price page [36]. Quantization does not explain the widest within-model spreads: in the widest panels the minimum and maximum prices are posted at comparable quantizations, and the largest spreads separate neocloud listings from hyperscaler managed offerings.

### 3.4 Statistical methods

To test H1 we compute each model’s median output price across providers (robust to loss-leaders and premium listings) and fit the hedonic regression (1) by ordinary least squares ( $n = 38$ ). The identifying assumption is that competing third parties’ posted prices reveal the cost structure of serving. This assumption fails for DeepSeek. The vendor lists its own models at strategically chosen prices, publicly commits to commoditizing the serving of its own models, and publishes its serving recipes [15, 16]. Third-party quotes for DeepSeek models are therefore anchored rather than competitive. We therefore estimate the schedule without the DeepSeek panels. Three robustness variants are reported: the full 46-model universe, the panel with every vendor’s own endpoints removed (which shows the exclusion is not doing hidden work), and the DeepSeek panels evaluated against the primary schedule (§5). We report standard errors,  $t$ -statistics, and  $p$ -values for every coefficient, and the regression  $F$ -statistic. To test H2 (§2.3) we proceed as before: we identify each panel’s floor-setting provider and its class, compare hyperscaler listings with panel floors, relate provider participation to model footprint, and measure within-model dispersion for MoE versus dense panels. All statistics are computed deterministically from the archived JSON by `analysis.py`, with no model-generated numbers.

## 4 Results

### 4.1 H1 rejected: the market prices memory footprint, not active compute

Figure 1 plots median output price against active parameters on log-log axes. The ten dense models trace a price line with a slope near unit elasticity ( $\log_{10} p = -1.691 + 0.950 \log_{10}(\text{active})$ ). Every MoE model sits above it, by  $1.0\times$  to  $21.6\times$  with a median of  $5.0\times$ , and the excess grows with the model’s sparsity ratio.

The regression quantifies the pattern (Table 1) and rejects H1 (§2.2) decisively. Compute pricing predicts  $c_2 = 0$ ; the estimate is 0.481 with  $t = 4.9$  and  $p = 1.9 \times 10^{-5}$ . The regression is jointly significant, with  $F(2, 35) = 16.8$  and  $p = 7.7 \times 10^{-6}$ , and explains about half of the variance in log prices ( $R^2 = 0.49$ ). The sparsity-ratio elasticity is 84% of the active-parameter elasticity:  $c_2/c_1 = 0.84$ . The sparsity ratio, which H1 predicts to be free, is in fact priced at 84% of the rate of active compute. The market bills MoE models most of the way toward their total memory footprint, so a 1,000B-total/32B-active model such as Kimi K2.6 is billed roughly like a  $\sim 580$ B dense model rather than the 32B model its compute would suggest.

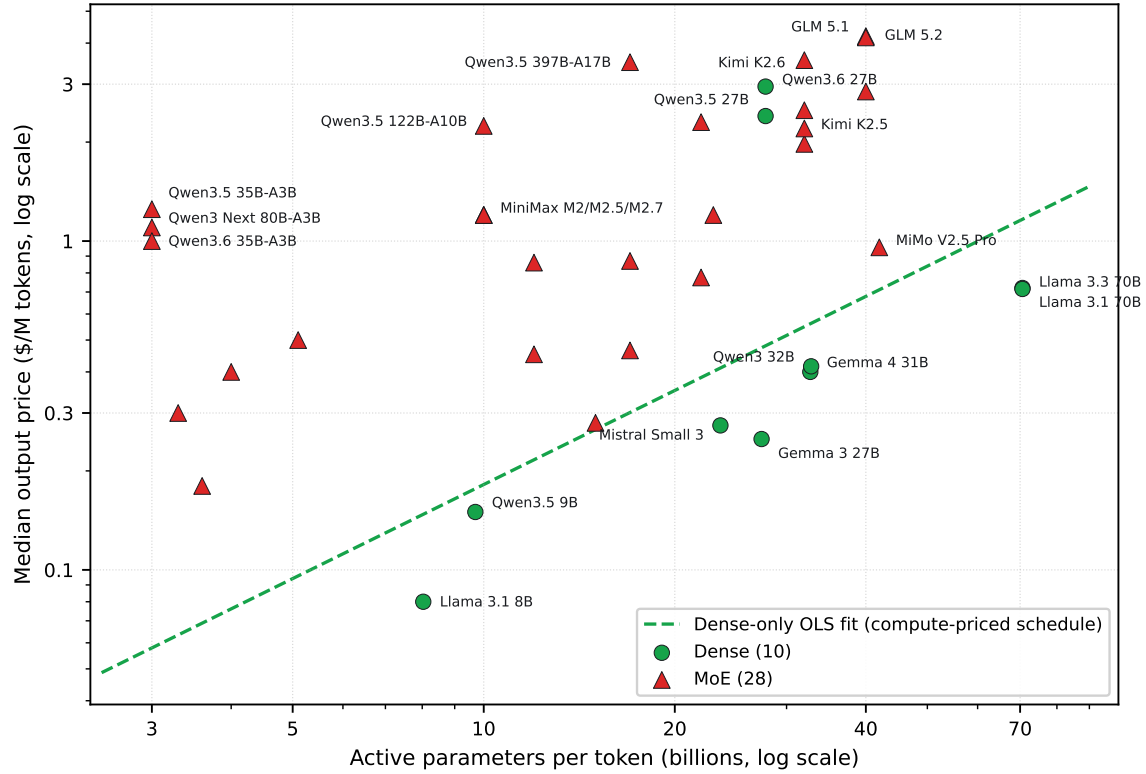


Figure 1: The figure plots median output price against active parameters per token for the 38 primary-panel models on log-log axes, with dense models as green circles and MoE models as red triangles. Labels follow a fixed rule: every dense model is labeled, as is every MoE model that sits at least six times above the dense-only line or carries at least 1,000B total parameters, and exactly coincident points share one label. The dashed line is the dense-only OLS fit, which is the price schedule a compute-priced market would apply. Every MoE model sits on or above the line, and the median MoE model sits  $5\times$  above it.

The result is robust to the construction of the panel. Including the DeepSeek panels moves the ratio to 0.85 ( $n = 46$ ,  $c_2 = 0.391$ ,  $p = 2.0 \times 10^{-4}$ ,  $R^2 = 0.37$ ). Removing every vendor’s own endpoints from every panel leaves it at 0.84 ( $n = 34$ ,  $c_2 = 0.468$ ,  $p = 1.3 \times 10^{-4}$ ,  $R^2 = 0.47$ ), which shows that the DeepSeek exclusion, and owner listings generally, are not driving the estimate.

Table 2 reports the underlying panel. Per active parameter, the MoE models cost up to  $40\times$  more than the dense reference (Qwen3.5 35B-A3B at \$0.42 per active-parameter-billion against  $\sim$ \$0.010 for the Llama dense models). The premium over the dense line tracks the sparsity ratio. The extremely sparse small MoE models (Qwen3.5 and Qwen3.6 35B-A3B, Qwen3 Next 80B-A3B) sit  $17\text{--}22\times$  above the line, the frontier flagships (Kimi K2.6, GLM 5.1/5.2) sit  $4\text{--}7\times$  above, and the mildly sparse Llama 4 Scout sits  $1.5\times$  above.

**Interpretation.** This result is a statement about what the market charges, and its stability across different ways of building the panel makes it hard to attribute to any single provider’s strategy. Under H1 the sparsity ratio should carry no price at all; instead it carries 84% of the price of active compute, with

Table 1: The table reports OLS estimates of the hedonic regression (1) on the primary panel ( $n = 38$  models,  $R^2 = 0.490$ ,  $F(2, 35) = 16.8$  with  $p = 7.7 \times 10^{-6}$ ). H1 (compute pricing) predicts  $c_2 = 0$ , and pure footprint pricing predicts  $c_2 = c_1$ ; the measured ratio is  $c_2/c_1 = 0.84$ .

| Coefficient                    | Estimate | Std. err. | $t$ | $p$                  |
|--------------------------------|----------|-----------|-----|----------------------|
| $c_1$ : log(active parameters) | +0.572   | 0.138     | 4.1 | $2.1 \times 10^{-4}$ |
| $c_2$ : log(total/active)      | +0.481   | 0.097     | 4.9 | $1.9 \times 10^{-5}$ |

a  $p$ -value below  $10^{-4}$  under every specification. The residual variance ( $R^2 = 0.49$ ) is itself structured. Newly released models (GLM 5.1/5.2, the Qwen3.5 and Qwen3.6 generations) list  $2\text{--}6\times$  above the price the fitted regression predicts for their size, which we call the *schedule price*. That is launch-premium pricing that has not yet been competed down. The DeepSeek panels, by contrast, sit far below the schedule price (§5).

## 4.2 H2 supported: the price floor belongs to the vertically integrated

Published deployments show a  $\sim 36\times$  per-GPU decode throughput gap between a large expert-parallel pool and a single 8-GPU node (Table 3), which is the scale mechanism H2 posits: no provider can sell at the pool’s marginal cost from a single-node deployment [16, 28, 15].

(a) *The price floor is set by vertically integrated providers.* In every one of the seventeen panels for models of  $\geq 200\text{B}$  total parameters, the lowest price is posted by a vertically integrated provider. The floor setter is a neocloud in eleven panels (DeepInfra in six, with Novita, StreamLake, Mara, ModelRun, and DigitalOcean setting the others) and the model’s own vendor in six (Xiaomi, MiniMax, and Alibaba each twice). No general-purpose cloud sets a floor in any panel (Table 4). DeepInfra, the most frequent floor setter, is the archetype of the class. It raised \$107M in Series B funding and owns and operates its GPU infrastructure across eight U.S. data centers. It publishes owned-hardware economics of  $\sim \$1.98$  per GPU-hour against \$6.50 for equivalent rented public-cloud capacity [37]. Figure 2 plots every provider quote in these panels and marks each panel’s floor.

(b) *Hyperscalers price above the floor and never set it.* Hyperscaler listings appear in five of the seventeen panels (Google Vertex and Amazon Bedrock; Microsoft Azure lists no open-weight endpoint on the marketplace at the snapshot date). Their premiums over the panel floor are  $1.5\times$  on GLM 5 (Bedrock),  $1.9\times$  on Llama 4 Maverick,  $1.3\times$  on GLM 4.7,  $1.2\times$  on MiniMax M2, and  $8.8\times$  on Qwen3-235B-A22B (all Google). All are positioned as managed enterprise offerings [38, 12].

## 4.3 Secondary result: price dispersion is compressed by selection

Within-model price dispersion remains compressed for MoE panels: the median coefficient of variation of output price is 0.23 for the 28 MoE panels against 0.35 for the 10 dense panels (Figure 3). The rank correlation between a panel’s expert-to-activation ratio  $E/k$  and its dispersion is negative but weaker than the floor result (Spearman  $\rho = -0.24$  over the 37 panels with published  $E/k$ ). The selection reading is unchanged in kind [39, 40]: providers whose latent cost exceeds the competitive price tend not to list, so the observed prices of the most scale-demanding models are a truncated sample. Participation in large-MoE serving has broadened in this snapshot (trillion-scale panels attract 11–22 providers as published expert-parallel recipes diffused [16, 15]), which weakens the truncation and is consistent with the weaker correlation.



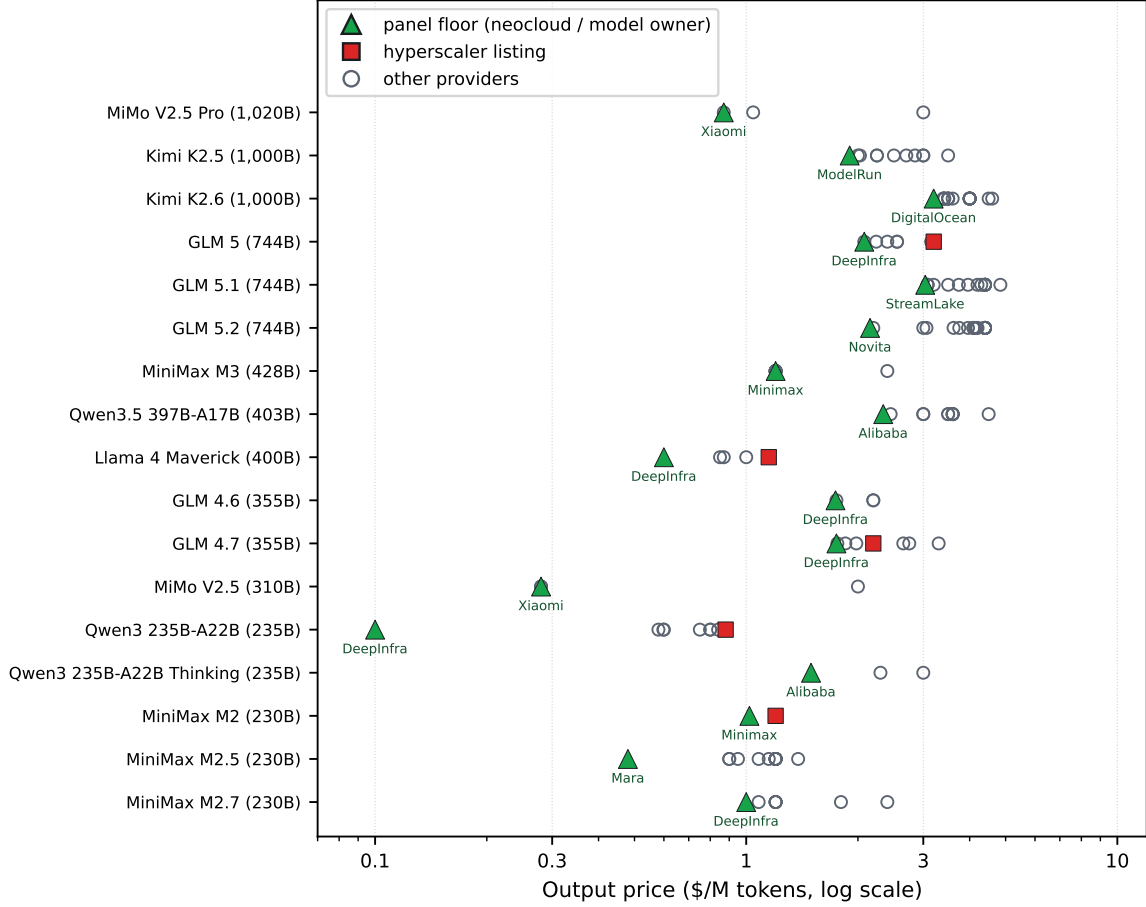


Figure 2: The figure plots every provider’s output price for the seventeen primary-panel models of 200B+ total parameters (log scale). The lowest quote in each panel (filled triangle, labeled) is posted by a neocloud in eleven panels and by the model’s owner in six. Hyperscaler listings (squares) never set a floor, and the remaining providers (open circles) fill the range between.

## 5 The DeepSeek exception

DeepSeek is excluded from the primary panel because its prices are set by a different process. To see what that process does, we take the regression fitted to the other 38 models and read off the price it predicts for each DeepSeek model, given that model’s active and total size. This predicted price is the schedule price. Table 5 sets it against what DeepSeek models actually sell for. Three regularities stand out.

First, the discount is generation-specific, which identifies it as technological rather than purely strategic. The pre-V3.2 models price near or moderately below the schedule (R1 at  $0.97\times$ , the V3.1 family at  $0.44\text{--}0.56\times$ ), while every model carrying DeepSeek’s sparse attention (V3.2, V3.2-Exp, V4 Flash) is served at roughly one fifth of the schedule price by a dozen competing third parties. The architecture genuinely lowers serving cost, and because DeepSeek publishes the serving recipes [15, 16], third parties can realize the saving and compete it into prices. The floor setters are the same class of

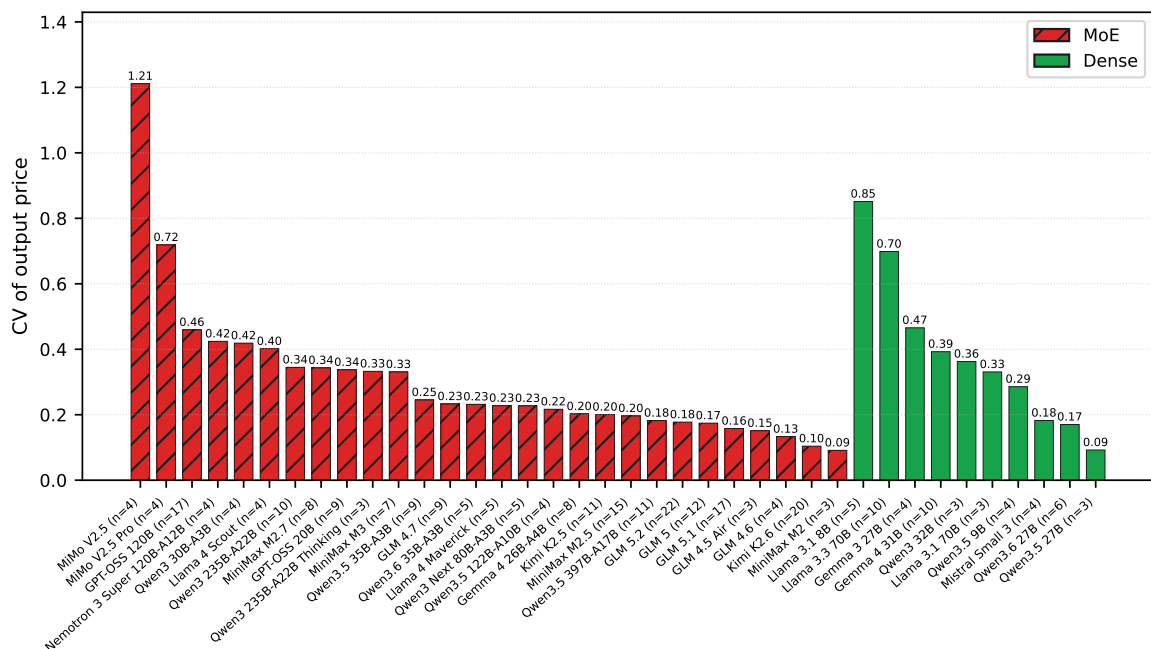


Figure 3: The figure shows within-model price dispersion, measured as the coefficient of variation of output price across providers, for MoE panels (hatched bars) versus dense panels (solid bars) in the primary panel.

provider that documents large-scale expert parallelism publicly [41].

Second, the vendor anchors. On its flagship V4 Pro, DeepSeek itself sets the floor at \$0.87 against a third-party median of \$3.14, underpricing the competitive market for its own model by  $3.6\times$ . This is the vertical-integration mechanism of H2 in its purest observed form: the one operator with hyperscale expert-parallel pools for a model prices it where no third party can follow.

Third, the exception clarifies rather than threatens our central finding, that the market prices a model by its total size rather than its active compute. Including the DeepSeek panels barely moves the elasticity ratio (0.85 against 0.84), because the pricing pattern comes from comparing many architectures and no single family can drive it. DeepSeek changes how tightly models cluster around that pattern, not the pattern itself. Its sparse-attention generation shows that the underlying cost of serving is falling, and that a vendor with the right strategy can push its prices away from what the rest of the market charges. A popular comparison from the snapshot week illustrates the point: DeepSeek V4 Flash (284B total) is served at a lower per-token price than Qwen3.6 35B-A3B (36B total), a model an eighth its size. Under any parameter-based schedule, including compute pricing, the larger model should cost more. The inversion is the combination of a genuine cost technology, published recipes, an owner anchor, and a launch premium on the smaller model, and it is visible only because both effects are large.

## 6 Discussion and limitations

Our evidence is observational and market-level: we observe posted prices and public listings, not provider costs, contracts, or deployments. Five limitations qualify the interpretation of the results. For

each, we state the limitation and the reason we believe the conclusions survive it.

- **Price is not cost.** List prices embed margins, loss-leader strategies, and enterprise positioning. The estimated elasticity establishes what the market *charges*; the inference to underlying cost is an interpretation. That interpretation is supported by the deployment-cost evidence of Table 3 (EP-pool cost \$0.20–0.50/M against large-MoE floors of \$0.28–3.20) and by the competitive structure of open-weight serving. LMSYS’s \$0.20/M and DeepSeek’s 545% margin figures, however, are self-reported, not audited.
- **Inference from prices is indirect.** Provider opacity is the premise of our design (§1), but it also limits what we can claim. That the floor-setting neoclouds run large-scale expert parallelism is inferred from their economics and from the public availability of EP serving software, not observed; only SiliconFlow documents an EP deployment [41]. No provider states publicly why it does or does not list a model, so the selective non-listing behind the dispersion result is inferred from documented economics rather than testimony. It is also unsurprising in its extreme form: that edge networks do not serve trillion-parameter models tells us little, which is one more reason participation is not used as a test. Our conclusions are consistency checks of the price structure against publicly reconstructable cost floors, not observations of provider internals.
- **Demand and quality are unobserved.** DeepSeek R1 commands  $2.1\times$  the price of its architecture-identical sibling V3, a reasoning-model demand premium that the regression absorbs as noise. Popularity may also correlate with aggressive pricing. For the same reason we do not read provider participation counts as a test of scale gating: newer, more capable models attract providers regardless of size, and the trillion-scale panels in this snapshot attract 11–22 providers.
- **The panel is a single snapshot.** It reflects one collection date (2026-07-06) in a market that reprices within weeks, and the residual structure shows time is not ignorable: newly released models list above the schedule and DeepSeek’s newest generation lists far below it. The collection pipeline is automated and mechanical, and we intend to re-collect monthly so that the schedule’s stability, and the decay of launch premia, can be estimated longitudinally.
- **The sample has limits.** OpenRouter covers the competitive open-weight market; hyperscaler-exclusive and closed models are underrepresented. The dense subsample is small ( $n = 10$ ) and confined to  $\leq 73\text{B}$ , because larger dense models have left the market, which is itself evidence of footprint economics. Delivered throughput tiers are not controlled. Finally, capacity and bandwidth requirements both scale with total parameters, so our test cannot distinguish “priced by bytes held” from “priced by bytes streamed”; under either reading, the priced quantity is memory, not compute.

## 7 Conclusion: implications for inference hardware

Two results emerge from the serving market’s own prices. First, the industry’s original claim for sparsity, that an MoE model costs what its active compute costs [9, 10, 11], is rejected: the market bills Mixture-of-Experts models by the memory they occupy, not the compute they perform. The sparsity elasticity of price is 84% of the active-parameter elasticity ( $p = 1.9 \times 10^{-5}$ ), so a customer of a 1,000B/32B model pays approximately as if for a  $\sim 580\text{B}$  dense model. Second, the price floor of every large-MoE panel belongs to a vertically integrated provider, a neocloud or the model’s own vendor, and general-purpose

clouds price  $1.2\text{--}8.8\times$  above the floor and never set it. DeepSeek’s sparse-attention generation shows the cost frontier itself moving, and the scale advantage shows most clearly in who sets the floor.

Both results point to the same hardware mismatch. The mainstream GPU bundles enormous compute with scarce, expensive HBM; MoE decoding uses the memory fully and the compute hardly at all, except inside hyperscale expert-parallel pools that only a few operators can build. For everyone else, the economically rational serving device is the opposite shape. It has **large, cheap memory capacity** to hold the full expert set without buying stranded FLOPS. It has **modest compute**, since decoding at realistic concurrency cannot exercise more. And it has **moderate bandwidth**, sufficient for interactive token rates and procurable from commodity supply chains rather than the allocation-constrained HBM market. Shipping products are beginning to approximate this design point, notably Apple’s Mac Studio (up to 512 GB unified LPDDR5) [42]. NVIDIA’s DGX Spark (128 GB LPDDR5X) [43] supplies the memory capacity but pairs it with a compute-to-bandwidth balance that only datacenter-scale concurrency could exercise, so it meets the capacity criterion alone. Purpose-built decode accelerators instantiate the full design point directly, such as the Skymizer HTX-301, a 28 nm PCIe card carrying 384 GB of commodity LPDDR, whose performance figures are vendor-claimed pending independent benchmarks [44]. The market prices measured here quantify the opportunity such hardware addresses. The gap between footprint-cheap serving cost and market price is widest for the most sparse frontier models (the Qwen 35B-A3B generations, Kimi K2.6, GLM 5.x), which is where the industry is heading.

**Data and reproducibility.** The raw data and analysis code are available at [https://github.com/juntao/memory\\_is\\_the\\_price](https://github.com/juntao/memory_is_the_price). The repository archives the price snapshot (`pd_prices_20260706.json`, 532 raw endpoint rows for 129 candidate models), the verified architecture data (`pd_arch_20260706.json`), and the collection script (`collect_snapshot.py`), which re-runs the candidate funnel live against OpenRouter’s API for any future snapshot. The analysis script (`analysis.py`) is dependency-free Python: OLS via normal equations, exact  $t$  and  $F$   $p$ -values via the incomplete beta function, and Spearman by ranks; the figures are generated by `make_figures.py`. Running `analysis.py` reproduces every statistic in §4–§5. All references below were verified by direct fetch at the time of writing; architecture parameters were verified against each repository’s `config.json`.

## References

- [1] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437. <https://arxiv.org/abs/2412.19437>
- [2] Meta. Llama-4-Scout-17B-16E-Instruct model repository ( $E=16$ ,  $k=1$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct>
- [3] Meta. Llama-4-Maverick-17B-128E-Instruct model repository ( $E=128$ ,  $k=1$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E-Instruct>
- [4] Qwen Team. Qwen3-235B-A22B-Instruct-2507 model repository, `config.json` ( $E=128$ ,  $k=8$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/Qwen/Qwen3-235B-A22B-Instruct-2507/raw/main/config.json>
- [5] Moonshot AI. Kimi-K2-Instruct model repository, `config.json` ( $E=384$ ,  $k=8$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/moonshotai/Kimi-K2-Instruct/raw/main/config.json>
- [6] OpenAI. gpt-oss-120b model repository, `config.json` ( $E=128$ ,  $k=4$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/openai/gpt-oss-120b/raw/main/config.json>

- [7] Z.ai. GLM-4.5 model repository, `config.json` (`n_routed_experts=160`, `k=8`). HuggingFace, fetched 2026-07-06. <https://huggingface.co/zai-org/GLM-4.5/raw/main/config.json>
- [8] MiniMax AI. MiniMax-M2 model repository ( $E=256$ ,  $k=8$ ; 230B total / 10B active). HuggingFace, fetched 2026-07-06. <https://huggingface.co/MiniMaxAI/MiniMax-M2>
- [9] Fedus, W., Zoph, B., & Shazeer, N. Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity. arXiv:2101.03961, 2021. <https://arxiv.org/abs/2101.03961> (“with outrageous numbers of parameters” at “a constant computational cost”).
- [10] Mistral AI. Mixtral of experts. 2023-12-11. <https://mistral.ai/news/mixtral-of-experts/> (“Mixtral has 46.7B total parameters but only uses 12.9B parameters per token. It, therefore, processes input and generates output at the same speed and for the same cost as a 12.9B model.”)
- [11] Databricks. Introducing DBRX: a new state-of-the-art open LLM. 2024-03-27. <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm> (DBRX inference throughput is “up to 2x faster” than Llama2-70B “thanks to having about half as many active parameters”).
- [12] Google. Llama 4 models generally available as managed API service on Vertex AI. Google Developers Blog, 2025-04-29. <https://developers.googleblog.com/en/llama-4-ga-maas-vertex-ai/> (“The underlying infrastructure, scaling, and management costs are incorporated into the API usage price.”). See also AWS Bedrock model card for DeepSeek-R1 (commercial terms only): <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-deepseek-deepseek-r1.html>
- [13] OpenRouter. Public endpoints API, per-model provider listings. <https://openrouter.ai/api/v1/models/{slug}/endpoints> (schema identical across all models). Snapshot collected 2026-07-06.
- [14] Gao, I., Liang, P., & Guestrin, C. Model equality testing: which model is this API serving? ICLR 2025. arXiv:2410.20247. <https://arxiv.org/abs/2410.20247>
- [15] DeepSeek-AI. DeepSeek-V3/R1 inference system overview (Open Infra Index, Day 6). [https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day\\_6\\_one\\_more\\_thing\\_deepseekV3R1\\_inference\\_system\\_overview.md](https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md)
- [16] LMSYS. Deploying DeepSeek with PD disaggregation and large-scale expert parallelism on 96 H100 GPUs. <https://www.lmsys.org/blog/2025-05-05-large-scale-ep/>
- [17] Cloudflare. Serving very large models on Workers AI (Infire; disaggregated prefill and expert parallelism). <https://blog.cloudflare.com/workers-ai-large-models/>; model catalog: <https://developers.cloudflare.com/workers-ai/models/>
- [18] Hayek, F. A. The use of knowledge in society. *American Economic Review* 35(4), 519–530, 1945.
- [19] Rosen, S. Hedonic prices and implicit markets: product differentiation in pure competition. *Journal of Political Economy* 82(1), 34–55, 1974.
- [20] Erdil, E. Inference economics of language models. Epoch AI. arXiv:2506.04645, June 2025. <https://arxiv.org/abs/2506.04645>. Bottom-up cost model over arithmetic, memory-bandwidth, network, and latency constraints; no market-price analysis.
- [21] Tensor Economics. MoE inference economics from first principles. <https://www.tensoreconomics.com/p/moe-inference-economics-from-first>

- [22] Li, H., et al. When is the same model not the same service? A measurement study of hosted open-weight LLM APIs. arXiv:2605.02821, May 2026. <https://arxiv.org/abs/2605.02821>. §4.4 (“Parameter Scale Is Not a Sufficient Price Model”) raises the active-vs-total pricing question descriptively.
- [23] Scher, A. Observations about LLM inference pricing. MIRI Technical Governance Team blog, 2025-03-03. <https://techgov.intelligence.org/blog/observations-about-llm-inference-pricing>. Fits dense price–parameter regressions and compares five MoE models against the dense trendline.
- [24] TrendForce. HBM3e pricing and DDR5 premium. <https://www.trendforce.com/presscenter/news/20251029-12758.html>
- [25] Silicon Analysts. AI accelerator cost bridge: HBM share of manufacturing cost. <https://siliconanalysts.com/tools/cost-bridge>
- [26] TechSpot. SK hynix sells out semiconductor supply into 2026. <https://www.techspot.com/news/110058-sk-hynix-completely-sells-out-semiconductor-supply-ai.html>
- [27] HPC-AI Tech. DeepSeek R1 671B deployment guide (8×H200 reference configuration). <https://company.hpc-ai.com/blog/deepseek-r1-671b-deployment-how-to-maximize-performance>
- [28] dzhsurf. DeepSeek V3/R1 deployment benchmarks, 8×H100 (AWQ 4-bit, vLLM). <https://github.com/dzhsurf/deepseek-v3-r1-deploy-and-benchmarks>
- [29] DeepSeek-AI. DeepGEMM: grouped GEMM kernels (BLOCK\_M = 128). <https://github.com/deepseek-ai/DeepGEMM>
- [30] Patel, P., Choukse, E., et al. (Microsoft Azure Research). Splitwise: efficient generative LLM inference using phase splitting. arXiv:2311.18677, 2023. <https://arxiv.org/abs/2311.18677>
- [31] Zhong, Y., et al. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. OSDI 2024. arXiv:2401.09670. <https://arxiv.org/abs/2401.09670>
- [32] Qin, R., et al. (Moonshot AI). Mooncake: a KVCache-centric disaggregated architecture for LLM serving. arXiv:2407.00079, 2024. <https://arxiv.org/abs/2407.00079>
- [33] NVIDIA. Scaling large MoE models with wide expert parallelism on NVL72 rack-scale systems. <https://developer.nvidia.com/blog/scaling-large-moe-models-with-wide-expert-parallelism-on-nvl72-rack-scale-systems/>
- [34] Artificial Analysis. Provider comparison pages. <https://artificialanalysis.ai/models/deepseek-r1/providers>
- [35] DeepSeek-AI. DeepSeek-R1-0528 model repository, `config.json` ( $E=256, k=8$ ). HuggingFace, fetched 2026-07-06. <https://huggingface.co/deepseek-ai/DeepSeek-R1-0528/raw/main/config.json>
- [36] DeepInfra. Qwen3-235B-A22B-Instruct-2507 price page (loss-leader verification). <https://deepinfra.com/Qwen/Qwen3-235B-A22B-Instruct-2507>
- [37] DeepInfra. Series B announcement and DeepCluster owned-hardware pricing. <https://deepinfra.com/blog/deepinfra-series-b>; <https://deepinfra.com/deepcluster>

- [38] Microsoft Azure AI Foundry, DeepSeek model pricing. <https://azure.microsoft.com/en-us/pricing/details/ai-foundry-models/deepseek/>; Amazon Web Services, DeepSeek-R1 on Bedrock. <https://aws.amazon.com/blogs/aws/deepseek-r1-now-available-as-a-fully-managed-serverless-model-in-amazon-bedrock/>
- [39] Stigler, G. J. The economics of information. *Journal of Political Economy* 69(3), 213–225, 1961.
- [40] Heckman, J. J. Sample selection bias as a specification error. *Econometrica* 47(1), 153–161, 1979.
- [41] Huawei & SiliconFlow. Serving large language models on Huawei CloudMatrix384. arXiv:2506.12708. <https://arxiv.org/pdf/2506.12708>
- [42] Apple. Apple reveals M3 Ultra, taking Apple silicon to a new extreme. Newsroom, March 2025. <https://www.apple.com/newsroom/2025/03/apple-reveals-m3-ultra-taking-apple-silicon-to-a-new-extreme/>
- [43] NVIDIA. DGX Spark. <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>
- [44] Skymizer Taiwan Inc. HTX-301 announcement (April 2026) and company profile v1.0 (March 2026). Performance figures vendor-claimed and pre-production.

Table 2: The table reports, for the 38-model primary panel, the median output price across providers (2026-07-06), the model architecture, the price per active-parameter-billion, and the multiple over the dense-only price line. Dense models, marked “–”, define the reference line.

| Model                      | Med \$/M | Act (B) | Tot (B) | \$/act-B | $\times$ line | $n$ |
|----------------------------|----------|---------|---------|----------|---------------|-----|
| GLM 5.1                    | 4.20     | 40      | 744     | 0.105    | 6.2           | 17  |
| GLM 5.2                    | 4.16     | 40      | 744     | 0.104    | 6.1           | 22  |
| Kimi K2.6                  | 3.55     | 32      | 1,000   | 0.111    | 6.5           | 20  |
| Qwen3.5 397B-A17B          | 3.50     | 17      | 403     | 0.206    | 11.6          | 11  |
| Qwen3.6 27B                | 2.95     | 27.8    | 27.8    | 0.106    | –             | 6   |
| GLM 5                      | 2.85     | 40      | 744     | 0.071    | 4.2           | 12  |
| Kimi K2.5                  | 2.50     | 32      | 1,000   | 0.078    | 4.6           | 11  |
| Qwen3.5 27B                | 2.40     | 27.8    | 27.8    | 0.086    | –             | 3   |
| Qwen3 235B-A22B Thinking   | 2.30     | 22      | 235     | 0.105    | 6.0           | 3   |
| Qwen3.5 122B-A10B          | 2.24     | 10      | 125     | 0.224    | 12.3          | 4   |
| GLM 4.7                    | 2.20     | 32      | 355     | 0.069    | 4.0           | 9   |
| GLM 4.6                    | 1.98     | 32      | 355     | 0.062    | 3.6           | 4   |
| Qwen3.5 35B-A3B            | 1.25     | 3       | 36      | 0.417    | 21.6          | 9   |
| MiniMax M2                 | 1.20     | 10      | 230     | 0.120    | 6.6           | 3   |
| MiniMax M2.5               | 1.20     | 10      | 230     | 0.120    | 6.6           | 15  |
| MiniMax M2.7               | 1.20     | 10      | 230     | 0.120    | 6.6           | 8   |
| MiniMax M3                 | 1.20     | 23      | 428     | 0.052    | 3.0           | 7   |
| Qwen3 Next 80B-A3B         | 1.10     | 3       | 81.3    | 0.367    | 19.0          | 5   |
| Qwen3.6 35B-A3B            | 1.00     | 3       | 36      | 0.333    | 17.3          | 5   |
| MiMo V2.5 Pro              | 0.96     | 42      | 1,020   | 0.023    | 1.3           | 4   |
| Llama 4 Maverick           | 0.87     | 17      | 400     | 0.051    | 2.9           | 5   |
| GLM 4.5 Air                | 0.86     | 12      | 106     | 0.072    | 4.0           | 3   |
| Qwen3 235B-A22B            | 0.78     | 22      | 235     | 0.035    | 2.0           | 10  |
| Llama 3.1 70B              | 0.72     | 70.6    | 70.6    | 0.010    | –             | 3   |
| Llama 3.3 70B              | 0.72     | 70.6    | 70.6    | 0.010    | –             | 10  |
| GPT-OSS 120B               | 0.50     | 5.1     | 117     | 0.098    | 5.2           | 17  |
| Llama 4 Scout              | 0.47     | 17      | 109     | 0.027    | 1.5           | 4   |
| Nemotron 3 Super 120B-A12B | 0.45     | 12      | 124     | 0.038    | 2.1           | 4   |
| Qwen3 32B                  | 0.42     | 32.8    | 32.8    | 0.013    | –             | 3   |
| Gemma 4 26B-A4B            | 0.40     | 4       | 26.5    | 0.100    | 5.3           | 8   |
| Gemma 4 31B                | 0.40     | 32.7    | 32.7    | 0.012    | –             | 10  |
| Qwen3 30B-A3B              | 0.30     | 3.3     | 30.5    | 0.091    | 4.7           | 4   |
| MiMo V2.5                  | 0.28     | 15      | 310     | 0.019    | 1.0           | 4   |
| Mistral Small 3            | 0.28     | 23.6    | 23.6    | 0.012    | –             | 4   |
| Gemma 3 27B                | 0.25     | 27.4    | 27.4    | 0.009    | –             | 4   |
| GPT-OSS 20B                | 0.18     | 3.6     | 21      | 0.050    | 2.6           | 9   |
| Qwen3.5 9B                 | 0.15     | 9.7     | 9.7     | 0.016    | –             | 4   |
| Llama 3.1 8B               | 0.08     | 8.0     | 8.0     | 0.010    | –             | 5   |



Table 3: The table compares decode throughput and cost across deployment scales. The  $\sim 36\times$  per-GPU gap between the EP pool and the single node is the mechanism behind H2. NVIDIA’s Wide-EP measurements on GB200 NVL72 (EP32 delivers up to  $1.8\times$  the per-GPU throughput of EP8) extend the pattern to current hardware [33].

| Deployment (DeepSeek R1/V3 class)                      | tok/s per GPU | \$/M out tokens                         | Source |
|--------------------------------------------------------|---------------|-----------------------------------------|--------|
| 8 $\times$ H100, single node (AWQ 4-bit, vLLM)         | $\sim 78$     | $\sim 7\text{--}10$ (market GPU rates)  | [28]   |
| 96 $\times$ H100, PD-disagg. + large-scale EP (SGLang) | $\sim 2,787$  | 0.20 (self-reported)                    | [16]   |
| DeepSeek production, EP144 decode (18-node H800)       | $\sim 1,850$  | $<0.50$ implied (545% margin at \$2.19) | [15]   |

Table 4: The table reports the floor-setting provider for each panel of 200B+ total parameters in the primary panel (2026-07-06).

| Panel (total params)       | Floor \$/M | Floor setter | Class       | $n$ |
|----------------------------|------------|--------------|-------------|-----|
| MiMo V2.5 Pro (1,020B)     | 0.87       | Xiaomi       | model owner | 4   |
| Kimi K2.5 (1,000B)         | 1.90       | ModelRun     | neocloud    | 11  |
| Kimi K2.6 (1,000B)         | 3.20       | DigitalOcean | neocloud    | 20  |
| GLM 5 (744B)               | 2.08       | DeepInfra    | neocloud    | 12  |
| GLM 5.1 (744B)             | 3.04       | StreamLake   | neocloud    | 17  |
| GLM 5.2 (744B)             | 2.16       | Novita       | neocloud    | 22  |
| MiniMax M3 (428B)          | 1.20       | MiniMax      | model owner | 7   |
| Qwen3.5 397B-A17B (403B)   | 2.34       | Alibaba      | model owner | 11  |
| Llama 4 Maverick (400B)    | 0.60       | DeepInfra    | neocloud    | 5   |
| GLM 4.6 (355B)             | 1.74       | DeepInfra    | neocloud    | 4   |
| GLM 4.7 (355B)             | 1.75       | DeepInfra    | neocloud    | 9   |
| MiMo V2.5 (310B)           | 0.28       | Xiaomi       | model owner | 4   |
| Qwen3 235B-A22B (235B)     | 0.10       | DeepInfra    | neocloud    | 10  |
| Qwen3 235B Thinking (235B) | 1.50       | Alibaba      | model owner | 3   |
| MiniMax M2 (230B)          | 1.02       | MiniMax      | model owner | 3   |
| MiniMax M2.5 (230B)        | 0.48       | Mara         | neocloud    | 15  |
| MiniMax M2.7 (230B)        | 1.00       | DeepInfra    | neocloud    | 8   |

Table 5: The table evaluates the eight DeepSeek panels against the primary schedule of Table 1. “Schedule” is the price the ex-DeepSeek regression predicts for the model’s architecture; “ratio” is the observed median over that prediction.

| Panel                           | Med \$/M | Schedule | Ratio | Floor setter        | $n$ |
|---------------------------------|----------|----------|-------|---------------------|-----|
| DeepSeek R1 (671/37)            | 2.18     | 2.24     | 0.97  | DeepInfra (\$2.15)  | 3   |
| DeepSeek V4 Pro (1,600/49)      | 3.14     | 3.49     | 0.90  | DeepSeek (\$0.87)   | 13  |
| DeepSeek V3.1 (671/37)          | 1.25     | 2.24     | 0.56  | DeepInfra (\$0.79)  | 6   |
| DeepSeek V3 (671/37)            | 1.06     | 2.24     | 0.47  | DeepInfra (\$0.90)  | 4   |
| DeepSeek V3.1 Terminus (671/37) | 0.98     | 2.24     | 0.44  | DeepInfra (\$0.95)  | 4   |
| DeepSeek V3.2 (671/37)          | 0.45     | 2.24     | 0.20  | StreamLake (\$0.34) | 12  |
| DeepSeek V3.2-Exp (671/37)      | 0.41     | 2.24     | 0.18  | Novita (\$0.41)     | 3   |
| DeepSeek V4 Flash (284/13)      | 0.28     | 1.35     | 0.21  | DeepInfra (\$0.18)  | 13  |